Reto Técnico – Ingeniero Cloud

BRYAN TOALUMBO RODRIGUEZ

1/03/2026

Introducción

El ejercicio plantea el diseño de una arquitectura para una aplicación web moderna compuesta por frontend, backend, base de datos y almacenamiento de objetos, considerando principios de alta disponibilidad, seguridad, escalabilidad y mantenibilidad.

La solución propuesta busca alinearse con escenarios reales de producción, utilizando servicios administrados del proveedor cloud seleccionado para reducir la carga operativa, mejorar la resiliencia del sistema y facilitar la evolución futura de la plataforma.

SUPOSICIONES TOMADAS

Dado que el enunciado no define ciertos aspectos funcionales y no funcionales, se asumieron los siguientes puntos para la propuesta de arquitectura:

- La aplicación es pública y accesible desde internet
- No se especifican restricciones de presupuesto, por lo que se prioriza diseño robusto y buenas prácticas por sobre optimización extrema de costos.
- La solución está pensada para ambiente productivo, no solo para pruebas.
- Se consideran buenas prácticas de seguridad, observabilidad y alta disponibilidad desde el diseño inicial.

RESPUESTAS TEÓRICAS

1. Diferencia entre nube pública, privada e híbrida

Nube Pública: Infraestructura operada por un proveedor externo ejemplo, AWS, Azure, Google Cloud, donde los recursos se comparten entre múltiples clientes.

Nube Privada: Recursos dedicados exclusivamente a una organización, ya sea on-premises o hospedados por un proveedor (single-tenant).

Nube Hibrida: Combina pública y privada, permitiendo mover datos/aplicaciones entre entornos mediante integración. Aprovecha lo mejor de ambos: seguridad privada para cargas críticas y escalabilidad pública para picos

2. Buenas prácticas de seguridad en la nube

En la nube, se prioriza la responsabilidad compartida donde el proveedor asegura la infraestructura y nosotros los datos y accesos. Entre las mejoras practicas de seguridad se presentas las siguientes:

- Gestión de Identidades y Accesos (IAM)
- Autenticación multifactor (MFA)
- Cifrado de Datos
- Segmentación de red (VPC, subredes privadas)
- Rotación de secretos y uso de Secret Managers

- Auditoría y logging centralizado
- Automatización de parches y escaneo de vulnerabilidades

3. ¿Qué es IaC, beneficios, 2 herramientas y características?

La IaC es una práctica DevOps clave que permite aprovisionar, configurar y gestionar infraestructura mediante archivos de código declarativo o imperativo, en lugar de procesos manuales o paneles web.

Beneficios:

- Repetibilidad
- Versionado
- Menos errores humanos
- Automatización CI/CD
- Trazabilidad de cambios

Herramientas:

- Terraform
- AWS CloudFormation

Características principales:

- Declarativo
- Idempotente
- Integrable con pipelines CI/CD
- Permite rollback de infraestructura

4. Métricas esenciales para monitoreo en la nube

Las métricas esenciales para monitoreo en la nube se centran en CPU, memoria, disco, red, disponibilidad, latencia y **costos**.

- Rendimiento del Sistema
- Disponibilidad (Uptime)
- Uso de CPU / Memoria
- Throughput
- Tiempo de respuesta de base de datos
- Tasa de requests
- **Costos y consumo de recursos**

5. ¿Qué es Docker y sus componentes principales?

Docker es una plataforma de contenedores que permite empaquetar aplicaciones con sus dependencias.

Componentes:

- Dockerfile: definición de la imagen

- Imagen: plantilla inmutable
- Contenedor: instancia en ejecución
- Docker Engine: motor de ejecución
- Docker Registry: repositorio de imágenes

6. Caso Práctico – Arquitectura Cloud

Proveedor elegido: **Amazon Web Services**

Justificación:

- Amplio ecosistema de servicios gestionados
- Alta disponibilidad global
- Excelente integración con contenedores
- Herramientas maduras de monitoreo y seguridad

Tecnologías requeridas

- Frontend: Angular / React (SPA)
- Reverse Proxy: Nginx en contenedor
- Backend: ASP.NET Core 10
- Load Balancer: Balanceador de carga
- Base de Datos: SQL Server Standard
- Almacenamiento de estáticos: S3
- Restricción clave:
  - Backend NO expuesto a internet
  - Base de datos solo accesible desde el backend
  - El frontend solo llama a /api vía Nginx

Componentes de la arquitectura:

Frontend (Angular / React)

SPA desplegado en:

- Amazon S3 (hosting estático)
- Distribuida por Amazon CloudFront

Load Balancer

- Application Load Balancer
- Expone solo HTTPS (443)
- Redirige tráfico al contenedor Nginx
- Termina TLS
- No expone directamente el backend

Reverse Proxy (Nginx en contenedor)

- Contenedor Nginx en Amazon ECS
- Función:

    - Recibe llamadas desde frontend a /api
    - Redirige internamente al backend
- Solo accesible desde el Load Balancer
- Backend NO tiene IP pública

Backend (ASP.NET Core 10)

- Contenedor Docker desplegado en Amazon ECS
- Subred privada
- Solo accesible desde Nginx
- Auto Scaling
- No expuesto a internet

Base de Datos (SQL Server Standard)

- Amazon RDS
- Motor: SQL Server Standard
- Subred privada
- Sin acceso público
- Security Group permite acceso solo desde backend

Almacenamiento de objetos (estáticos)

- Amazon S3
- Imágenes, assets, contenido estático
- Acceso público solo vía CloudFront
- Cifrado habilitado

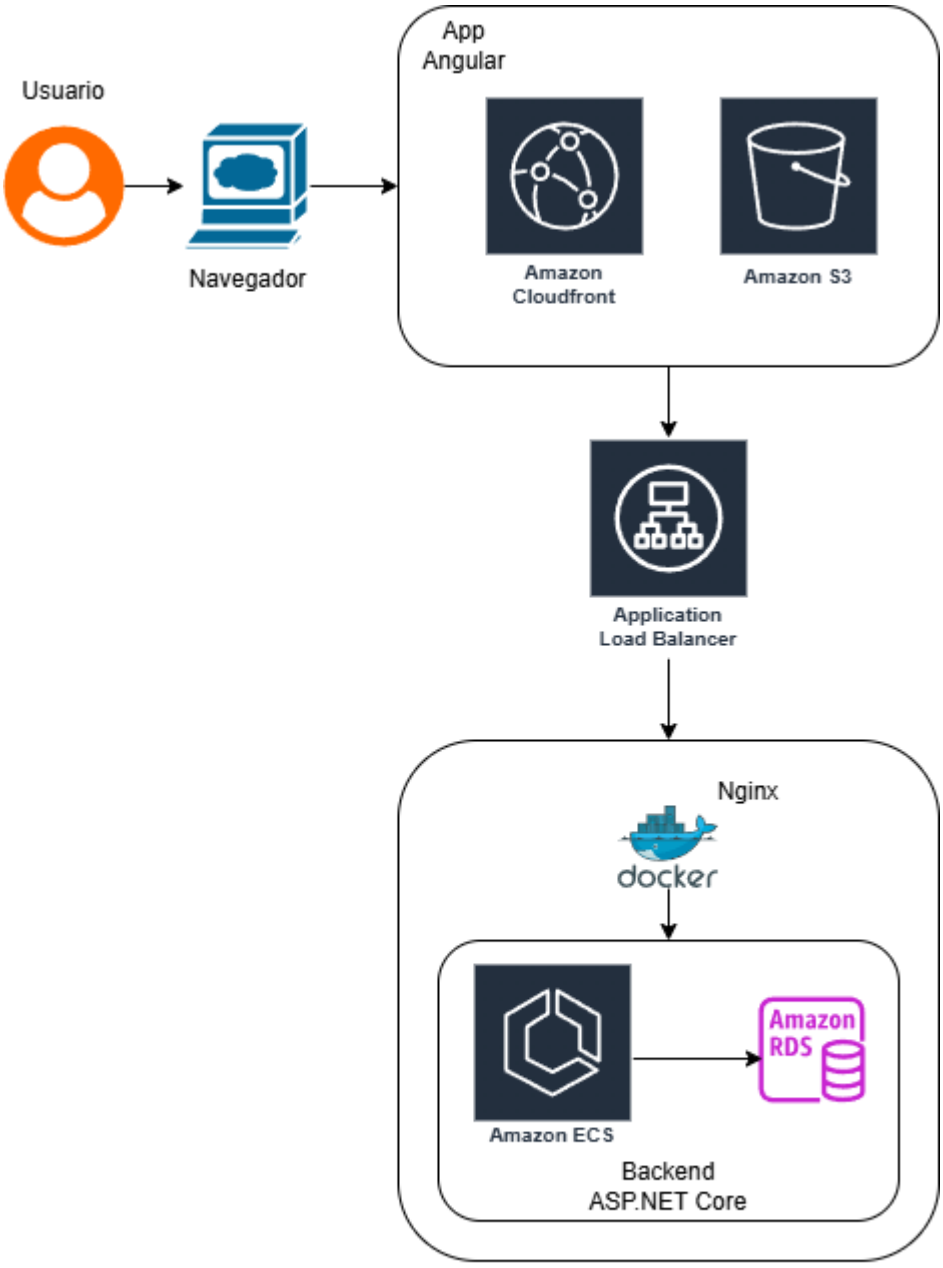
Seguridad de la arquitectura

- Backend sin IP pública
- DB en subred privada
- Security Groups:
    - ALB → Nginx
    - Nginx → Backend
    - Backend → RDS
- HTTPS obligatorio
- Secretos en AWS Secrets Manager
- Accesos mínimos con AWS IAM
- Logs centralizados con Amazon CloudWatch

Justificación de la arquitectura:

La arquitectura replica un patrón de entrada única (single entry point) común en entornos empresariales, donde CloudFront y el Load Balancer actúan como frontera de seguridad, y el reverse proxy Nginx protege el acceso a los servicios internos.

Este enfoque permite escalar, proteger y evolucionar la plataforma sin exponer directamente componentes críticos, alineándose con principios de Zero Trust y defensa en profundidad.



URL: https://github.com/Bryan47749/reto-tecnico-devsu